



Universidad Veracruzana

Implementación de roles en la aplicación logística

Autorización con grupos de Django

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito de la práctica

Implementar autorización por roles en el proyecto de logística usando grupos de Django.

El cliente podrá registrar y consultar sus propios bienes; el supervisor podrá consultar todos los bienes registrados.

Problema a resolver

La aplicación ya permite trabajar con bienes, pero necesita controlar quién puede verlos y quién puede registrarlos.

Cliente
└─ registra y consulta sus propios bienes

Supervisor
└─ consulta todos los bienes

Usuario sin sesión
└─ no accede a vistas protegidas

Decisión arquitectónica

La funcionalidad de clientes se implementará como app de dominio.

La lógica transversal de autorización se mantendrá en el módulo `seguridad` .

<code>apps/clientes/</code>	→ modelo Cliente
<code>apps/bienes/</code>	→ modelo Bien y vistas de bienes
<code>seguridad/</code>	→ roles y decoradores

Estructura esperada

```
logistica/  
├── apps/  
│   ├── bienes/  
│   │   ├── models.py  
│   │   ├── forms.py  
│   │   ├── views.py  
│   │   ├── urls.py  
│   │   └── templates/bienes/  
│   └── clientes/  
│       ├── models.py  
│       ├── admin.py  
│       └── apps.py  
├── seguridad/  
│   ├── __init__.py  
│   ├── roles.py  
│   └── decorators.py  
└── config/  
    ├── settings.py  
    └── urls.py
```

Resultado esperado

Al finalizar, el proyecto tendrá:

- app `clientes` ;
- modelo `Cliente` relacionado con `User` ;
- modelo `Bien` relacionado con `Cliente` ;
- grupos `cliente` y `supervisor` ;
- funciones auxiliares en `seguridad/roles.py` ;
- decoradores en `seguridad/decorators.py` ;
- vistas protegidas;
- reportes diferenciados por rol.

Paso 1. Crear la app clientes

Desde la raíz del proyecto:

```
python manage.py startapp clientes apps/clientes
```

La estructura generada será:

```
apps/clientes/  
├── __init__.py  
├── admin.py  
├── apps.py  
├── migrations/  
├── models.py  
├── tests.py  
└── views.py
```

Paso 2. Ajustar apps.py

```
# apps/clientes/apps.py

from django.apps import AppConfig

class ClientesConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "apps.clientes"
```

Esto permite que Django ubique correctamente la app dentro de la carpeta `apps` .

Paso 3. Registrar la app en settings.py

```
# config/settings.py

INSTALLED_APPS = [
    # Apps de Django
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Apps del proyecto
    "apps.clientes",
    "apps.bienes",
    "apps.rastreo",
]
```

El módulo `seguridad` no se registra porque no es una app Django.

Paso 4. Crear el modelo Cliente

```
# apps/clientes/models.py

from django.db import models
from django.contrib.auth.models import User

class Cliente(models.Model):
    usuario = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name="cliente"
    )
    identificador = models.CharField(max_length=30, unique=True)
    nombre = models.CharField(max_length=120)
    apellido_paterno = models.CharField(max_length=120)
    apellido_materno = models.CharField(max_length=120)
    correo = models.EmailField()
    telefono = models.CharField(max_length=20)
    creado_en = models.DateTimeField(auto_now_add=True)
    actualizado_en = models.DateTimeField(auto_now=True)

    @property
    def nombre_completo(self):
        return (
            f"{self.nombre} "
            f"{self.apellido_paterno} "
            f"{self.apellido_materno}"
        )

    def __str__(self):
        return f"{self.identificador} - {self.nombre_completo}"
```

Explicación del modelo Cliente

Campo	Propósito
usuario	Relaciona el cliente con una cuenta de Django
identificador	Clave funcional del cliente
nombre	Nombre del cliente
apellido_paterno	Primer apellido del cliente
apellido_materno	Segundo apellido del cliente
correo	Medio de contacto electrónico
telefono	Medio de contacto telefónico
creado_en	Fecha de creación
actualizado_en	Fecha de última modificación

Relación entre User y Cliente

Django ya tiene usuarios para iniciar sesión.

El modelo `Cliente` representa la información de negocio asociada a ese usuario.

```
User
└─ Cliente
    └─ Bienes
```

La relación con `User` permite saber qué cliente inició sesión y qué bienes le pertenecen.

Paso 5. Registrar Cliente en admin

```
# apps/clientes/admin.py

from django.contrib import admin
from .models import Cliente

@admin.register(Cliente)
class ClienteAdmin(admin.ModelAdmin):
    list_display = (
        "identificador",
        "nombre",
        "apellido_paterno",
        "apellido_materno",
        "correo",
        "telefono",
        "usuario",
        "creado_en",
        "actualizado_en",
    )
    search_fields = (
        "identificador",
        "nombre",
        "apellido_paterno",
        "apellido_materno",
        "correo",
    )
    list_filter = ("creado_en",)
```

Uso del administrador

En esta práctica, el administrador se puede usar para:

- crear usuarios;
- crear grupos;
- asignar usuarios a grupos;
- registrar clientes;
- asociar cada cliente con un usuario.

El administrador se usa como apoyo técnico para preparar los datos de prueba.

Paso 6. Actualizar el modelo Bien

El modelo `Bien` debe pertenecer a un cliente.

```
# apps/bienes/models.py

from django.db import models
from apps.clientes.models import Cliente

class Bien(models.Model):
    ESTATUS_CHOICES = [
        ("bueno", "Bueno"),
        ("regular", "Regular"),
        ("malo", "Malo"),
    ]

    cliente = models.ForeignKey(
        Cliente,
        on_delete=models.CASCADE,
        related_name="bienes"
    )
    identificador = models.CharField(max_length=30, unique=True)
    descripcion = models.TextField()
    marca = models.CharField(max_length=80)
    valor = models.DecimalField(max_digits=10, decimal_places=2)
    estatus = models.CharField(max_length=20, choices=ESTATUS_CHOICES)
    creado_en = models.DateTimeField(auto_now_add=True)
    actualizado_en = models.DateTimeField(auto_now=True)

    def __str__(self):
```

Por qué Bien se relaciona con Cliente

La autorización necesita saber a quién pertenece cada bien.

```
User → Cliente → Bien
```

Esto permite aplicar reglas como:

```
Bien.objects.filter(cliente__usuario=request.user)
```

Sin propiedad del dato, no se puede filtrar con seguridad por usuario.

Paso 7. Crear migraciones

```
python manage.py makemigrations clientes  
python manage.py makemigrations bienes  
python manage.py migrate
```

Si ya existen bienes previos, Django puede solicitar un valor por defecto para el nuevo campo `cliente`.

Para la práctica, se recomienda trabajar con una base limpia o crear clientes antes de registrar bienes.

Paso 8. Crear grupos cliente y supervisor

Los grupos pueden crearse desde el administrador de Django.

```
Admin Django
→ Authentication and Authorization
→ Groups
→ Add group
```

Crear:

```
cliente
supervisor
```

Después, asignar cada usuario al grupo correspondiente.

Alternativa: crear grupos desde shell

```
python manage.py shell
```

```
from django.contrib.auth.models import Group  
  
Group.objects.get_or_create(name="cliente")  
Group.objects.get_or_create(name="supervisor")
```

Esta opción permite preparar el ambiente sin depender del panel administrativo.

Paso 9. Crear funciones de roles

```
# seguridad/roles.py

GRUPO_CLIENTE = "cliente"
GRUPO_SUPERVISOR = "supervisor"

def pertenece_a_grupo(user, nombre_grupo):
    return user.is_authenticated and user.groups.filter(
        name=nombre_grupo
    ).exists()

def es_cliente(user):
    return pertenece_a_grupo(user, GRUPO_CLIENTE)

def es_supervisor(user):
    return pertenece_a_grupo(user, GRUPO_SUPERVISOR)
```

Paso 10. Crear decoradores

```
# seguridad/decorators.py

from functools import wraps
from django.core.exceptions import PermissionDenied
from .roles import es_cliente, es_supervisor

def supervisor_requerido(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not es_supervisor(request.user):
            raise PermissionDenied("Se requiere rol de supervisor.")
        return view_func(request, *args, **kwargs)
    return wrapper

def cliente_requerido(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not es_cliente(request.user):
            raise PermissionDenied("Se requiere rol de cliente.")
        return view_func(request, *args, **kwargs)
    return wrapper
```

Paso 11. Crear formulario de Bien

```
# apps/bienes/forms.py

from django import forms
from .models import Bien

class BienForm(forms.ModelForm):
    class Meta:
        model = Bien
        fields = [
            "identificador",
            "descripcion",
            "marca",
            "valor",
            "estatus",
        ]
```

El campo `cliente` no se incluye en el formulario porque se asignará desde el usuario autenticado.

Por qué no incluir cliente en el formulario

Si el cliente pudiera seleccionar el campo `cliente`, podría intentar registrar bienes a nombre de otro usuario.

Incorrecto:
el usuario elige a qué cliente pertenece el bien.

Correcto:
el servidor asigna el cliente según `request.user`.

Los datos sensibles de autorización deben decidirse en el backend.

Paso 12. Vista para registrar bienes

```
# apps/bienes/views.py

from django.contrib.auth.decorators import login_required
from django.shortcuts import redirect, render

from seguridad.decorators import cliente_requerido

from .forms import BienForm

@login_required
@cliente_requerido
def crear_bien(request):
    cliente = request.user.cliente

    if request.method == "POST":
        form = BienForm(request.POST)

        if form.is_valid():
            bien = form.save(commit=False)
            bien.cliente = cliente
            bien.save()
            return redirect("reporte_mis_bienes")
    else:
        form = BienForm()

    return render(
        request,
        "bienes/crear_bien.html",
        {"form": form}
    )
```


Qué protege la vista de alta

Control	Propósito
@login_required	Exige sesión activa
@cliente_requerido	Solo permite registrar a clientes
request.user.cliente	Asigna propiedad desde el backend
form.save(commit=False)	Permite agregar el cliente antes de guardar
Sin campo cliente en formulario	Evita manipulación de propietario

Paso 13. Vista reporte de mis bienes

```
# apps/bienes/views.py

from django.contrib.auth.decorators import login_required
from django.shortcuts import render

from seguridad.decorators import cliente_requerido
from .models import Bien

@login_required
@cliente_requerido
def reporte_mis_bienes(request):
    bienes = Bien.objects.filter(
        cliente__usuario=request.user
    ).order_by("-creado_en")

    return render(
        request,
        "bienes/reporte_mis_bienes.html",
        {"bienes": bienes}
    )
```

Paso 14. Vista supervisor

```
# apps/bienes/views.py

from django.contrib.auth.decorators import login_required
from django.shortcuts import render

from seguridad.decorators import supervisor_requerido
from .models import Bien

@login_required
@supervisor_requerido
def reporte_general_bienes(request):
    bienes = Bien.objects.select_related("cliente").order_by("-creado_en")

    return render(
        request,
        "bienes/reporte_general_bienes.html",
        {"bienes": bienes}
    )
```

Paso 15. Vista lista según rol

```
# apps/bienes/views.py

from django.contrib.auth.decorators import login_required
from django.shortcuts import render

from seguridad.roles import es_cliente, es_supervisor
from .models import Bien

@login_required
def lista_bienes(request):
    if es_supervisor(request.user):
        bienes = Bien.objects.select_related("cliente").all()

    elif es_cliente(request.user):
        bienes = Bien.objects.filter(
            cliente__usuario=request.user
        )

    else:
        bienes = Bien.objects.none()

    return render(
        request,
        "bienes/lista_bienes.html",
        {
            "bienes": bienes,
            "es_cliente": es_cliente(request.user),
            "es_supervisor": es_supervisor(request.user),
        }
    )
```

Paso 16. Configurar rutas

```
# apps/bienes/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path("bienes/", views.lista_bienes, name="lista_bienes"),
    path("bienes/nuevo/", views.crear_bien, name="crear_bien"),
    path(
        "bienes/mis-bienes/",
        views.reporte_mis_bienes,
        name="reporte_mis_bienes"
    ),
    path(
        "bienes/reporte-general/",
        views.reporte_general_bienes,
        name="reporte_general_bienes"
    ),
]
```

Paso 17. Integrar rutas del proyecto

```
# config/urls.py

from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("apps.bienes.urls")),
]
```

Si `clientes` no tiene vistas propias, no necesita rutas por ahora.

Paso 18. Template para crear bien

```
<!-- apps/bienes/templates/bienes/crear_bien.html -->

<h1>Registrar bien</h1>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Guardar</button>
</form>

<a href="{% url 'reporte_mis_bienes' %}">
    Ver mis bienes
</a>
```

Este formulario usa `{% csrf_token %}` porque lo envía una persona desde navegador.

Paso 19. Template mis bienes

```
<!-- apps/bienes/templates/bienes/reporte_mis_bienes.html -->

<h1>Mis bienes registrados</h1>

<a href="{% url 'crear_bien' %}">Registrar nuevo bien</a>

<table>
  <thead>
    <tr>
      <th>Identificador</th>
      <th>Descripción</th>
      <th>Marca</th>
      <th>Valor</th>
      <th>Estatus</th>
    </tr>
  </thead>
  <tbody>
    {% for bien in bienes %}
      <tr>
        <td>{{ bien.identificador }}</td>
        <td>{{ bien.descripcion }}</td>
        <td>{{ bien.marca }}</td>
        <td>{{ bien.valor }}</td>
        <td>{{ bien.estatus }}</td>
      </tr>
    {% empty %}
      <tr>
        <td colspan="5">No hay bienes registrados.</td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```


Paso 20. Template reporte general

```
<!-- apps/bienes/templates/bienes/reporte_general_bienes.html -->

<h1>Reporte general de bienes</h1>

<table>
  <thead>
    <tr>
      <th>Cliente</th>
      <th>Identificador</th>
      <th>Descripción</th>
      <th>Marca</th>
      <th>Valor</th>
      <th>Estatus</th>
    </tr>
  </thead>
  <tbody>
    {% for bien in bienes %}
      <tr>
        <td>{{ bien.cliente.nombre_completo }}</td>
        <td>{{ bien.identificador }}</td>
        <td>{{ bien.descripcion }}</td>
        <td>{{ bien.marca }}</td>
        <td>{{ bien.valor }}</td>
        <td>{{ bien.estatus }}</td>
      </tr>
    {% empty %}
      <tr>
        <td colspan="6">No hay bienes registrados.</td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```

Paso 21. Template lista según rol

```
<!-- apps/bienes/templates/bienes/lista_bienes.html -->

<h1>Bienes</h1>

{% if es_cliente %}
  <a href="{% url 'crear_bien' %}">Registrar bien</a>
  <a href="{% url 'reporte_mis_bienes' %}">Mis bienes</a>
{% endif %}

{% if es_supervisor %}
  <a href="{% url 'reporte_general_bienes' %}">
    Reporte general
  </a>
{% endif %}

<table>
  <thead>
    <tr>
      <th>Identificador</th>
      <th>Descripción</th>
      <th>Marca</th>
      <th>Valor</th>
      <th>Estatus</th>
    </tr>
  </thead>
  <tbody>
    {% for bien in bienes %}
      <tr>
        <td>{{ bien.identificador }}</td>
        <td>{{ bien.descripcion }}</td>
        <td>{{ bien.marca }}</td>
        <td>{{ bien.valor }}</td>
        <td>{{ bien.estatus }}</td>
      </tr>
    {% empty %}
      <tr>
        <td colspan="5">No hay bienes registrados.</td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```

Paso 22. Configurar login

```
# config/settings.py

LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "lista_bienes"
LOGOUT_REDIRECT_URL = "login"
```

Si se usan las vistas de autenticación de Django, integrar rutas:

```
# config/urls.py

urlpatterns = [
    path("admin/", admin.site.urls),
    path("cuentas/", include("django.contrib.auth.urls")),
    path("", include("apps.bienes.urls")),
]
```

Rutas de autenticación disponibles

Con `django.contrib.auth.urls`, Django proporciona rutas como:

Ruta	Uso
<code>/cuentas/login/</code>	Inicio de sesión
<code>/cuentas/logout/</code>	Cierre de sesión
<code>/cuentas/password_change/</code>	Cambio de contraseña
<code>/cuentas/password_reset/</code>	Restablecimiento

Para esta práctica se usará principalmente login y logout.

Paso 23. Crear usuarios de prueba

Crear tres usuarios:

Usuario	Grupo	Cliente asociado
cliente_a	cliente	Cliente A
cliente_b	cliente	Cliente B
supervisor_1	supervisor	No requiere cliente

Los usuarios con rol `cliente` deben tener registro en el modelo `cliente` .

Ejemplo de clientes de prueba

Identificador	Nombre	Apellido paterno	Apellido materno	Usuario
CLI-0001-2026	Ana	López	Martínez	cliente_a
CLI-0002-2026	Luis	Hernández	Pérez	cliente_b

Estos clientes permitirán probar que cada usuario solo consulta sus propios bienes.

Paso 24. Flujo de prueba

1. Crear usuarios

Alta de usuarios y grupos.

2. Asociar clientes

Crear Cliente A y Cliente B.

3. Registrar bienes

Cada cliente registra sus bienes.

4. Revisar reportes

Cliente ve propios; supervisor ve todos.

Pruebas obligatorias

Caso	Resultado esperado
Usuario sin sesión abre /bienes/	Redirección a login
Cliente A registra un bien	Se guarda asociado a Cliente A
Cliente A consulta mis bienes	Solo ve bienes de Cliente A
Cliente B consulta mis bienes	Solo ve bienes de Cliente B
Supervisor consulta reporte general	Ve bienes de todos
Cliente abre reporte general	Acceso denegado

Prueba de manipulación

El estudiante debe comprobar que el cliente no puede elegir manualmente el propietario del bien.

El campo `cliente` no debe aparecer en el formulario.

El cliente inicia sesión.

Registra un bien.

El servidor asigna automáticamente `request.user.cliente`.

La propiedad del bien se asigna en el backend, no desde el formulario.

Errores frecuentes

Error	Consecuencia
Registrar <code>seguridad</code> en <code>INSTALLED_APPS</code>	Confusión arquitectónica
Incluir <code>cliente</code> en el formulario	Riesgo de manipulación
No asociar <code>Cliente</code> con <code>User</code>	No se identifica el propietario
No filtrar por <code>request.user</code>	El cliente ve bienes ajenos
No usar <code>@login_required</code>	Acceso sin sesión
Usar solo templates para ocultar enlaces	Acceso posible por URL

Entregables

Los estudiantes deberán entregar:

- estructura actualizada del proyecto;
- modelo `Cliente` con los campos definidos;
- modelo `Bien` actualizado;
- archivo `seguridad/roles.py` ;
- archivo `seguridad/decorators.py` ;
- formulario `BienForm` ;
- vistas de alta y reportes;
- templates básicos;
- evidencia de pruebas por rol.

Criterios de evaluación

Criterio	Evidencia
App clientes creada	apps/clientes registrada
Cliente vinculado a User	Relación OneToOneField
Entidad Cliente completa	Campos definidos para el proyecto
Bien vinculado a Cliente	Relación ForeignKey
Roles implementados	Grupos cliente y supervisor
Alta protegida	Solo cliente registra bienes
Reporte cliente	Solo muestra bienes propios
Reporte supervisor	Muestra todos los bienes
Seguridad backend	No depende solo del template

Síntesis

La autorización por roles en Django se logra combinando sesión activa, grupos, funciones auxiliares, decoradores y consultas filtradas por propietario.

En el proyecto de logística, esto permite separar con claridad lo que puede hacer un cliente y lo que puede consultar un supervisor.

Gracias
¿Preguntas?

